

# sol-pay-client — API specification

Status: draft, revised 2026-08-31. Written before implementation; sections 6.2 onward describe code that does not exist yet.

This specifies the client library that a site integrates. It is the companion to `state-machine.plantuml` at the repository root, which remains the authoritative description of the flow. Where this document and the diagram disagree, the diagram is right and this document is a bug.

## 1. What this is

A library, not an application. sol-pay ships no user interface. The cyan nodes in the state diagram – `sign_up`, `account_page`, `metered_page` – are screens the *integrator* builds. They appear in the design only to establish what the library owes them:

1. the data needed to render each screen, and
2. the operations its controls invoke.

Nothing else. The library does not route, render, format, or decide.

**Out of scope, decided 2026-08-31: gifting a page view by sharing a link.** The rationale is in §4.4.

## 2. The design rule

**Be rigid where the chain is rigid. Be silent where the site has a legitimate choice.**

An integrator rejects a library that dictates their product. They keep one that stops them shipping bugs. Everything below is sorted by that rule.

### Chain facts — the library is authoritative

Wrong here costs a failed transaction and a real fee, so the library owns these completely and no integrator should reimplement them:

- instruction encoding, account order, and Anchor discriminators
- PDA derivation for site, contract, and slug index
- account layout and decoding
- the rule that `approve` must precede `open_contract` / `renew_contract` in the same transaction
- the arithmetic that decides whether a meter call will succeed
- that `approve` *replaces* an allowance rather than adding to it
- the mapping from an on-chain error code to a cause the UI can branch on

### Site policy — the library has no view

- what limit to suggest or offer
- how to format an amount, or in what currency to display it
- when to show an account page, redirect, or block
- whether metering is per view, per article, or batched
- **how the server identifies the visitor** (§4)
- whether sign-in precedes sign-up
- what to do when a payment fails

If a future API would decide any of the above, it does not belong here.

**The line, stated once: describe the chain's state, do not prescribe the site's response.**

## 3. Two consumers

A site running sol-pay executes code in two places, with different needs.

|         | signs                     | needs                                                                                                                                                                                             | artifact        |
|---------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| Browser | payer, via wallet adapter | <code>approve_checked</code> ,<br><code>open_contract</code> ,<br><code>renew_contract</code> ,<br><code>close_contract</code> ,<br><code>revoke</code> , decoders,<br>preflight, sign-in message | npm package     |
| Server  | site authority            | <code>meter_and_settle</code> ,<br><code>initialize_site</code> ,<br>decoders, preflight,<br>sign-in verification                                                                                 | crates.io crate |

The server never holds the payer’s key; the browser never holds the site authority. `meter_and_settle` is the only instruction the site signs, and the program enforces it (`has_one = authority`).

## 4. Session resolution

This section exists because the library must not force a site to abandon the login it already has.

### 4.1 The slug is not in the payment path

`meter_and_settle` takes the payer’s address and derives the contract from `[b"contract", site, payer]`. Its eight accounts include no slug index. The slug appears only in `open_contract`, `renew_contract`, and `close_contract`, and there only to maintain the index that maps a slug to a contract.

So the slug does not make payment work. It answers a different question, and the two are separable:

- **Who is this visitor?** — session resolution. Site’s choice.
- **What do they owe?** — the payment core. The library’s job.

The payment core needs exactly one thing from the integrator: **the payer’s wallet address**. How the site learned it is not the library’s business.

### 4.2 Three strategies, none privileged

**A. The site’s existing session.** A site with accounts, login, or SSO already authenticates its users. It adds a wallet address to the user record and is done. No slug, no new session mechanism, nothing to migrate. This is expected to be the common case for an established publisher, and it needs nothing from the library beyond the payment core.

**B. Wallet sign-in.** The visitor proves control of the wallet, the site holds an authenticated session. See §6.6. This is the option for a site with no existing account system that wants one.

**C. Slug in the URL.** No server-side session state at all. A random token in the path resolves to the contract in one derive plus one account read. This is the strategy the state diagram draws, and the library implements it, but it is one option and not the default posture.

### 4.3 What strategy C actually buys

The original motivation was avoiding a cookie-consent banner. That motivation is weaker than it looks: authentication session cookies are the textbook “strictly necessary” case under the ePrivacy Directive and need no banner, provided they are used *only* for authentication and are session cookies rather than persistent “remember me” tokens. A site that logs users in is already banner-free on that count.

The claim that survives is narrower and more interesting: **strategy C needs no server-side session state whatsoever**. That is a real architectural property and worth offering. It is not a legal shortcut, and this document should not be read as legal advice.

### 4.4 Sharing is out of scope

A slug in a URL is a bearer token for the payer’s *entire remaining balance*. Sharing a link does not gift an article; it gifts everything up to the limit, for as long as the slug lives.

The familiar publisher feature – NYT-style gift links – is a different mechanism: one-time, expiring, scoped to a single article. If gifting is ever wanted, it should be built in that shape, where the exposure is one article rather than a balance.

Consequence: **the slug is a secret, not a shareable token**. That is a change in kind, and it makes the storage defect in §9.2 a live problem rather than an accepted trade-off.

## 5. Two published artifacts

Decided 2026-08-31.

**sol-pay-client on crates.io** — the `core` module as a normal Rust crate, for a site's Rust server. Requires flipping the feature default: today `default = ["wasm"]`, which would drag `wasm-bindgen` into a server build. It becomes `default = []` with `wasm` opt-in.

**An npm package** — the `wasm-pack --target web` output: the `.wasm`, the JS glue, and the generated `.d.ts`.

One source tree, one version number.

Carried over from the dependency policy: a published crate's `Cargo.lock` is ignored by consumers. They re-resolve inside the ranges in `Cargo.toml`, so those ranges become the real compatibility contract on the day this ships.

## 6. API surface

### 6.1 Write path — exists

Instruction builders in `core::ix`, address derivation in `core::pda`, slug encoding in `core::slug`. Unchanged by this spec. Every builder stays public so an integrator can compose transactions their own way.

### 6.2 Read path — `core::state`, to be written

Nothing in the library reads chain state today, so `account_page` cannot be rendered by a consumer at all: its “show current limit, cost per page, used and paid amount” spans the `Contract` and `Site` accounts and we export no way to decode either.

Layouts in field order, each preceded by an 8-byte Anchor account discriminator (`sha256("account:<Name>") [. . 8]`):

| account   | fields                                                                                                                       | bytes |
|-----------|------------------------------------------------------------------------------------------------------------------------------|-------|
| Site      | authority [u8;32], mint [u8;32],<br>treasury [u8;32], page_price u64,<br>collection_threshold u64,<br>min_limit u64, bump u8 | 129   |
| Contract  | site [u8;32], payer [u8;32], slug<br>[u8;16], limit u64, used u64, paid<br>u64, bump u8, slug_bump u8                        | 114   |
| SlugIndex | contract [u8;32], bump u8                                                                                                    | 41    |

```
impl Site    { pub fn decode(data: &[u8]) -> Result<Self, DecodeError>; }
impl Contract { pub fn decode(data: &[u8]) -> Result<Self, DecodeError>;
                pub fn unpaid(&self) -> u64;           // used - paid
                pub fn outstanding(&self) -> u64; } // limit - paid
impl SlugIndex { pub fn decode(data: &[u8]) -> Result<Self, DecodeError>; }
```

```
/// SPL mint decimals, one byte at a fixed offset. `approve_checked` needs it
/// and there is nowhere else to get it without decoding a mint by hand.
pub fn mint_decimals(mint_account_data: &[u8]) -> Result<u8, DecodeError>;
```

`decode` verifies the discriminator and length before reading. Sizes are not written as literals: the parity test asserts them against the program's `INIT_SPACE`, so a field added on chain fails a test here.

### 6.3 Preflight — `core::preflight`, to be written

The choice nodes in the diagram are arithmetic over `Contract` and `Site`, and the minimum-limit rule appears at four places in the flow. If each integrator reimplements them they drift from the program, and the drift surfaces as a rejected transaction the payer paid fees for.

These return **facts, not instructions**. Nothing here says what to render.

```
pub fn charge(site: &Site, page_views: u32) -> Option<u64>;
pub fn can_meter(c: &Contract, s: &Site, page_views: u32) -> Result<(), Blocked>;
pub fn will_settle(c: &Contract, s: &Site, page_views: u32) -> bool;
pub fn views_remaining(c: &Contract, s: &Site) -> u64;
pub fn min_open_limit(s: &Site) -> u64;
pub fn min_renewal_limit(c: &Contract, s: &Site) -> u64;
pub fn required_allowance(limit: u64) -> u64;
```

Each mirrors one on-chain check exactly:

- `can_meter` -> `used + page_price * views <= limit`, else `LimitReached`
- `will_settle` -> `used + charge - paid >= collection_threshold`
- `min_open_limit` -> `site.min_limit`
- `min_renewal_limit` -> `max(site.min_limit, contract.unpaid())`, which is both renewal requirements at once: at or above the site minimum, and covering usage carried forward
- `required_allowance` -> the full limit; nothing is paid against a new limit yet, so the allowance must cover all of it

### 6.4 Errors — `core::error`, to be written

An integrator currently receives an Anchor code and a log string, and must match on text to tell “limit reached” (route to renewal) from “insufficient funds” (tell the payer their wallet is short).

```
pub enum PayError { /* the program's nine variants, in declaration order */ }
impl PayError {
    pub fn from_code(code: u32) -> Option<Self>;
    pub fn code(&self) -> u32; // 6000 + ordinal
    pub fn message(&self) -> &'static str;
}
```

Anchor numbers `#[error_code]` variants from 6000 in declaration order. That is an Anchor implementation detail, so the parity test pins each variant against the program’s own `PayError` as `u32` rather than trusting the offset.

### 6.5 Ordered transactions — `core::tx`, to be written

“The approve must precede the program instruction” is currently a paragraph in a README. It is the rule an integrator gets wrong once and then debugs for an hour, and the program deliberately refuses to trust the client on it.

```
pub fn open_contract_tx(..) -> Vec<Instruction>; // approve_checked, open_contract
pub fn renew_contract_tx(..) -> Vec<Instruction>; // approve_checked, renew_contract
pub fn close_contract_tx(..) -> Vec<Instruction>; // close_contract, revoke
```

Convenience, not a gate: the individual builders stay public and nothing is reachable only through these. They exist so the correct thing is also the shortest thing to write.

`close_contract_tx` pairs the close with a revoke. The leftover approval is inert once the contract account is gone – the PDA can no longer sign – but it stays visible in the payer’s wallet until withdrawn.

### 6.6 Wallet sign-in — `core::signin`, to be written

Strategy B from §4.2, and the mechanism that makes `sign_in` in the diagram an authorization rather than a public read.

Sign In With Solana is a `signIn` feature in the Wallet Standard. The site builds a structured message – domain, address, statement, uri, version, chainId, nonce, issuedAt, expirationTime, requestId, resources – the wallet displays and signs it, and the server verifies the signature against the message it issued.

```
// browser package
pub fn sign_in_input(domain: &str, uri: &str, nonce: &str, ..) -> SignInInput;

// native crate, server side
pub fn verify_sign_in(issued: &SignInInput, signed: &SignInOutput)
    -> Result<Pubkey, SignInError>;
```

Two integration facts the library should absorb rather than leave to the integrator:

- **Not every wallet has the feature.** Phantom’s documentation notes support from Phantom extension 23.11.0 onward. Where `signIn` is absent the fallback is connect plus `signMessage` over the same message bytes, and verification is identical. The library builds the message either way.
- **The nonce is the site’s to generate and to remember**, and expiry must be checked at verification. A verifier that ignores `expirationTime` accepts a replayed sign-in forever.

Scope question in §9.4: verification needs `ed25519`, a dependency this crate does not currently carry.

## 6.7 JavaScript surface

Every item above gets a `wasm_bindgen` wrapper on the existing convention: camelCase `js_name`, base58 strings for addresses, `JsError` for failures.

One constraint: `u64` amounts must cross as `BigInt`, not `number`. A JS number loses precision above  $2^{53}$ , and while USDC balances will not reach it, a library that silently truncates is not one an integrator can audit.

## 7. What the library never does

Signing, RPC, retries, storage, routing, rendering, session management. It builds instructions, decodes bytes, and verifies a signature. The integrator owns the wallet adapter, the connection, the session, and the page.

This is the boundary integrators get wrong, so it is stated here and repeated in the README.

## 8. Drift control

Every claim this document makes about the program is pinned by a test in `pay-on-chain/tests`, the one place both sides build together:

- instruction bytes against Anchor’s generated types (exists)
- the hardcoded program id against `declare_id!` (exists)
- account discriminators against `sha256("account:<Name>")`
- decoded account sizes against `INIT_SPACE`
- each `PayError` variant against the program’s discriminant
- preflight predicates against the program’s behaviour: for a case a predicate calls blocked, the matching `LiteSVM` call must actually fail, with the same error

The last one is the point of the exercise. A predicate that disagrees with the program is worse than no predicate.

## 9. Open questions

### 9.1 Program id: fixed or overridable?

The id is compiled in, so a published package is bound to one deployment. The `Site` PDA is seeded by authority, so one deployment already serves many sites with independent pricing – which suggests a single canonical deployment is the intended model and the fixed id is a feature. But an integrator wanting their own deployment then cannot use the published package at all. An override defaulting to the canonical id costs little and removes a reason to reject the library.

### 9.2 The slug is stored in the clear — program change proposed

`Contract.slug` is a plain field in a public account, and the contract PDA is derived from `[b"contract", site, payer]` where both seeds are public. Every contract is enumerable from the program’s transaction history or one `getProgramAccounts` on the discriminator, and each hit yields a working slug.

While sharing was in scope this was a trade-off. With sharing out of scope (§4.4) it is a defect: under strategy C, a third party can consume a payer’s balance to the limit, obtaining metered content the payer pays for. The

funds reach the site treasury, so this is theft of service rather than theft of funds, and a content scraper is a likelier actor than a vandal.

Proposed fix, not yet applied, and it touches merged on-chain code:

- drop `slug` (and `slug_bump`) from `Contract`
- take the current slug as an *argument* to `renew_contract` and `close_contract`
- derive the slug index from that argument, and require `slug_index.contract == contract.key()`

That last constraint proves the caller holds the contract's real slug, which is exactly what the seed check proves today, so nothing is weakened. No stored hash is needed.

Cost: sign-in can no longer recover a lost slug from chain state. Acceptable – strategies A and B identify the payer without a slug, and a strategy C site whose payer loses the slug can renew, which rotates it.

Until this is applied, strategy C's exposure must be documented for integrators, and the only available mitigation is rate limiting at the site's request layer – a mitigation, not a fix, and defeated by a distributed client.

### 9.3 Does WASM discourage adoption; should a plain-JS package exist?

Raised, deferred, not analysed.

### 9.4 Ship SIWS verification, or document it?

Verification needs ed25519 signature checking. The core crate currently carries almost no dependencies, and adding one to a payment library is not free. The alternatives are to implement it, or to specify the message format precisely and point integrators at `@solana/wallet-standard-util` in JS and an existing `siws` crate in Rust. Implementing it fits the “easy to get right” goal; documenting it fits the “small and auditable” one.